THOTH SYSTEM ARCHITECTURE

Production Multi-Agent Systems Engineering & Epistemic Verification

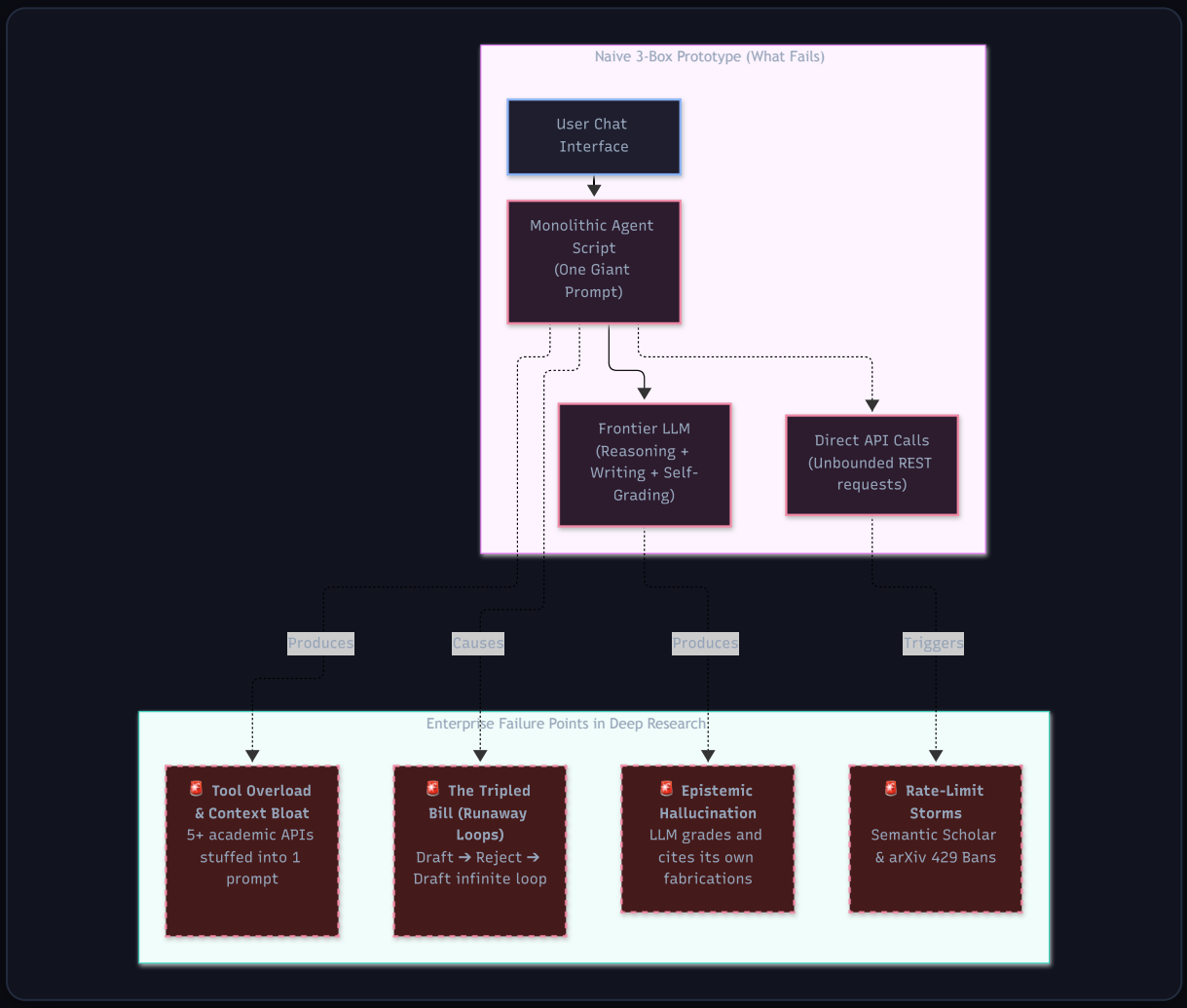
SYSTEM SPECIFICATION v2.0 | VECTOR 8K MERMAID SUITE | FAIL-CLOSED INTEGRITY GATE | EPISTEMIC ENGINE: POPPER / BACON

Thoth System Architecture: Step-by-Step Production Design

This architecture suite breaks down Thoth into 6 discrete, modular steps inspired by the production-hardening framework from Sanjay Kumar's *Demo to Production* breakdown.

Step 1: The Naive Baseline vs. The Production Research Crisis

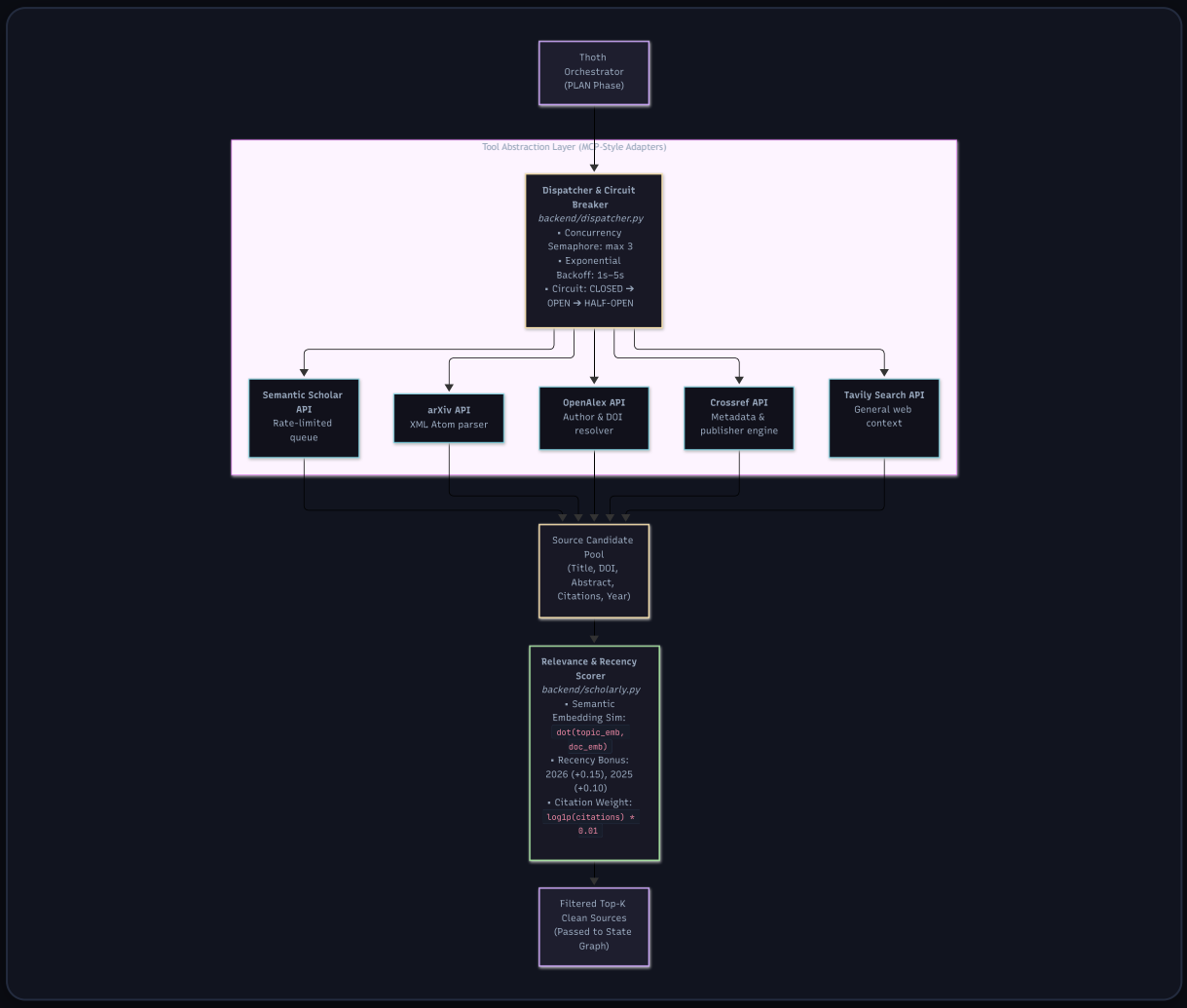
The contrast between a basic 3-box demo and the failure modes that necessitate a multi-agent distributed system.



diagram

Step 2: Tool Abstraction & Resilient Harvesting (The MCP & Dispatcher Pattern)

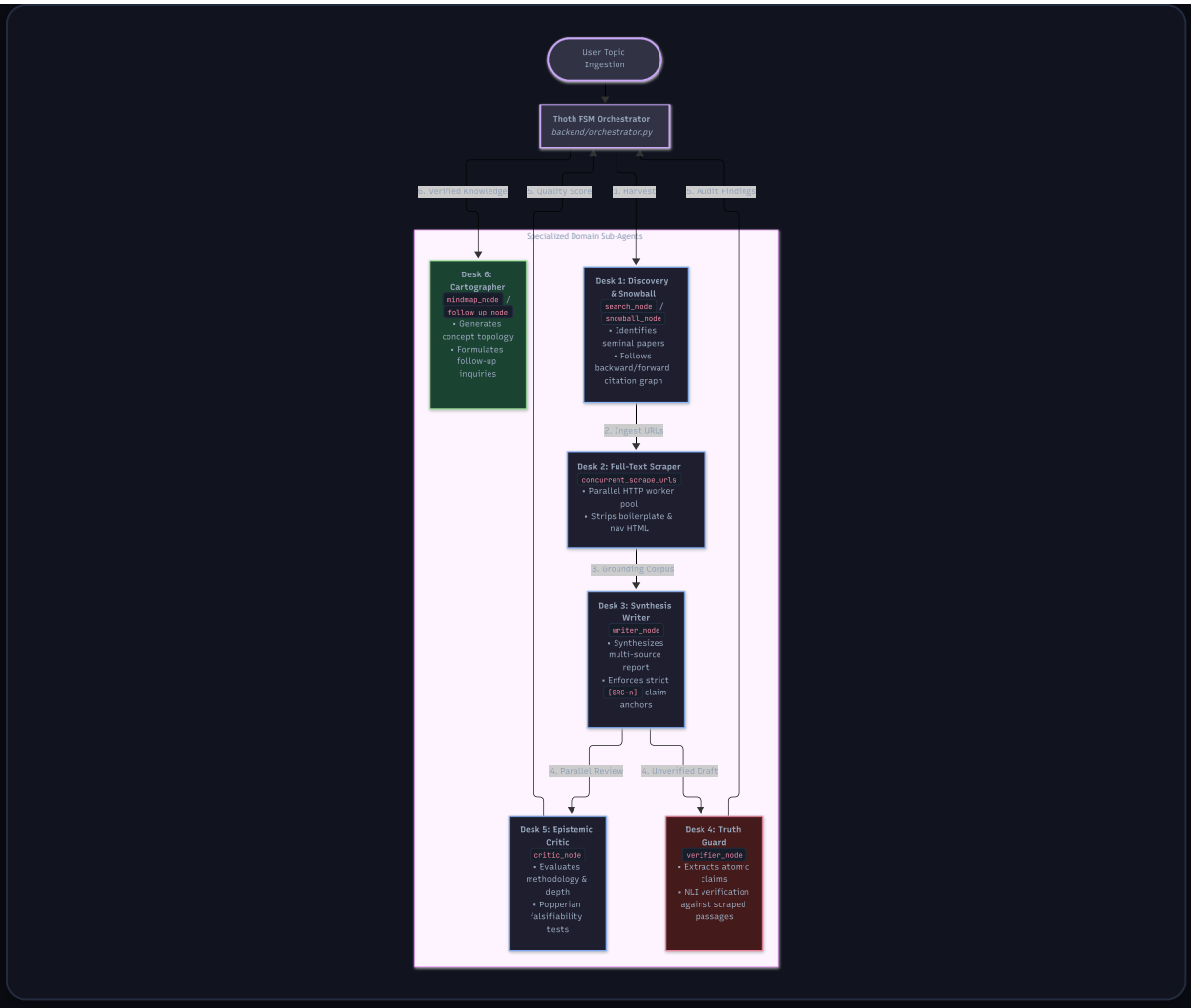
Abstracting academic endpoints behind circuit breakers, token budgets, and concurrency limiters.



diagram

Step 3: Multi-Agent Decomposition (Coordinator + Specialized Desks)

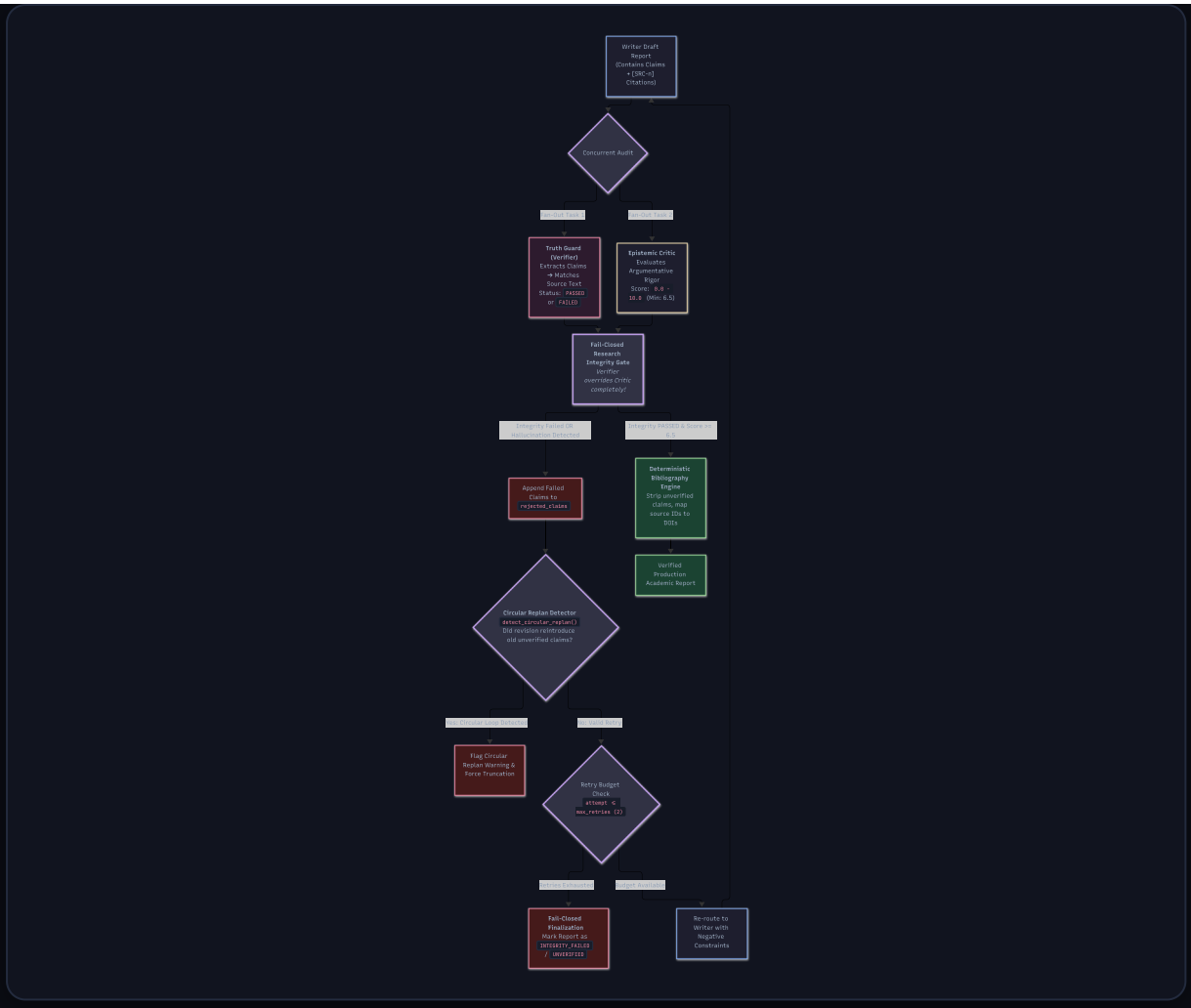
Decoupling research tasks into small, single-responsibility sub-agents supervised by a deterministic Finite State Machine.



diagram

Step 4: Dual-Key Verification & Defense (The Security & Gate Layer)

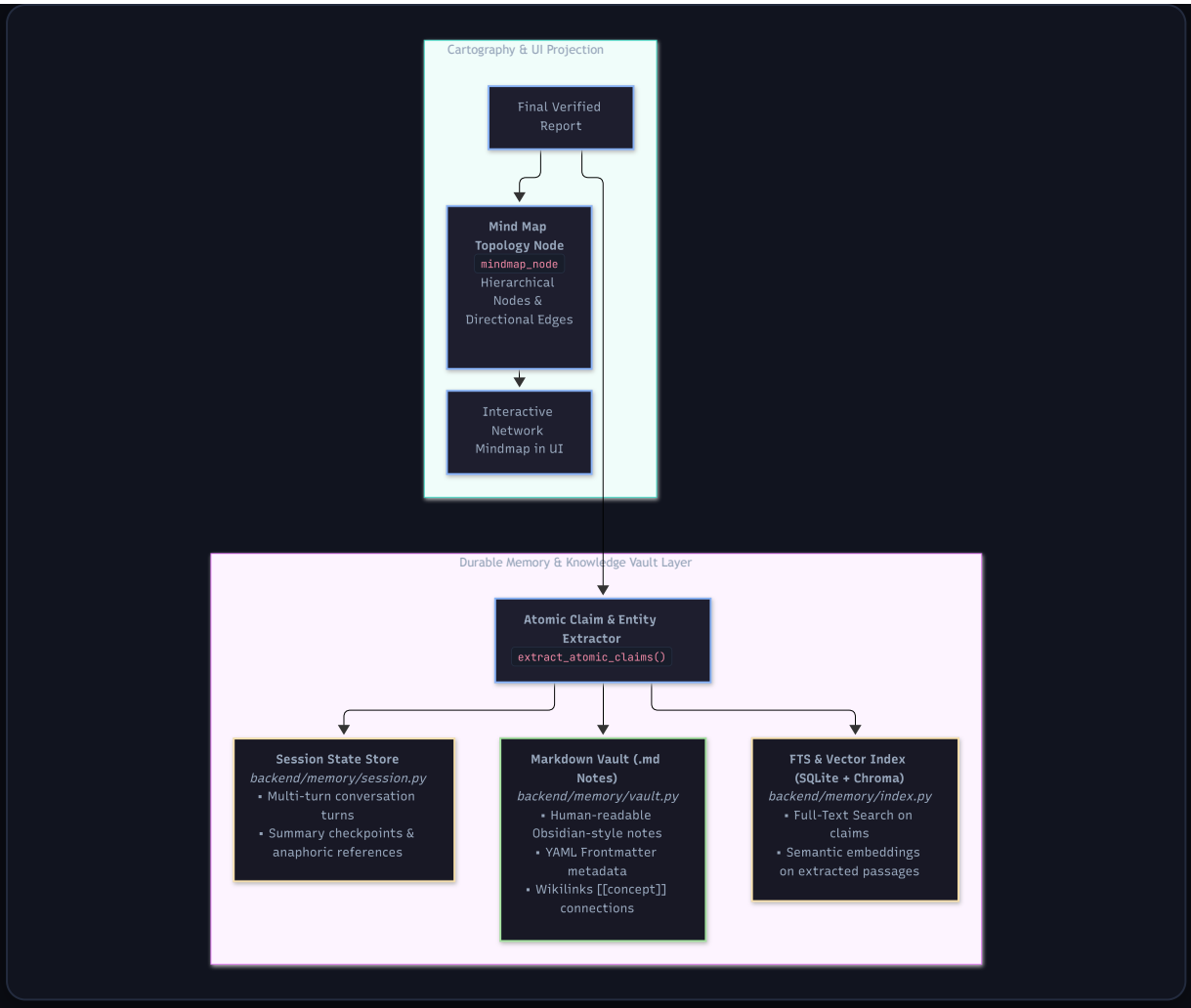
The fail-closed verification gate preventing hallucinations and circular revision loops.



diagram

Step 5: Persistent State & Long-Term Memory (The Knowledge Vault)

Separating ephemeral prompt tokens from durable, queryable external storage.



diagram

Step 6: Operational Reliability, Observability & Circuit Breakers

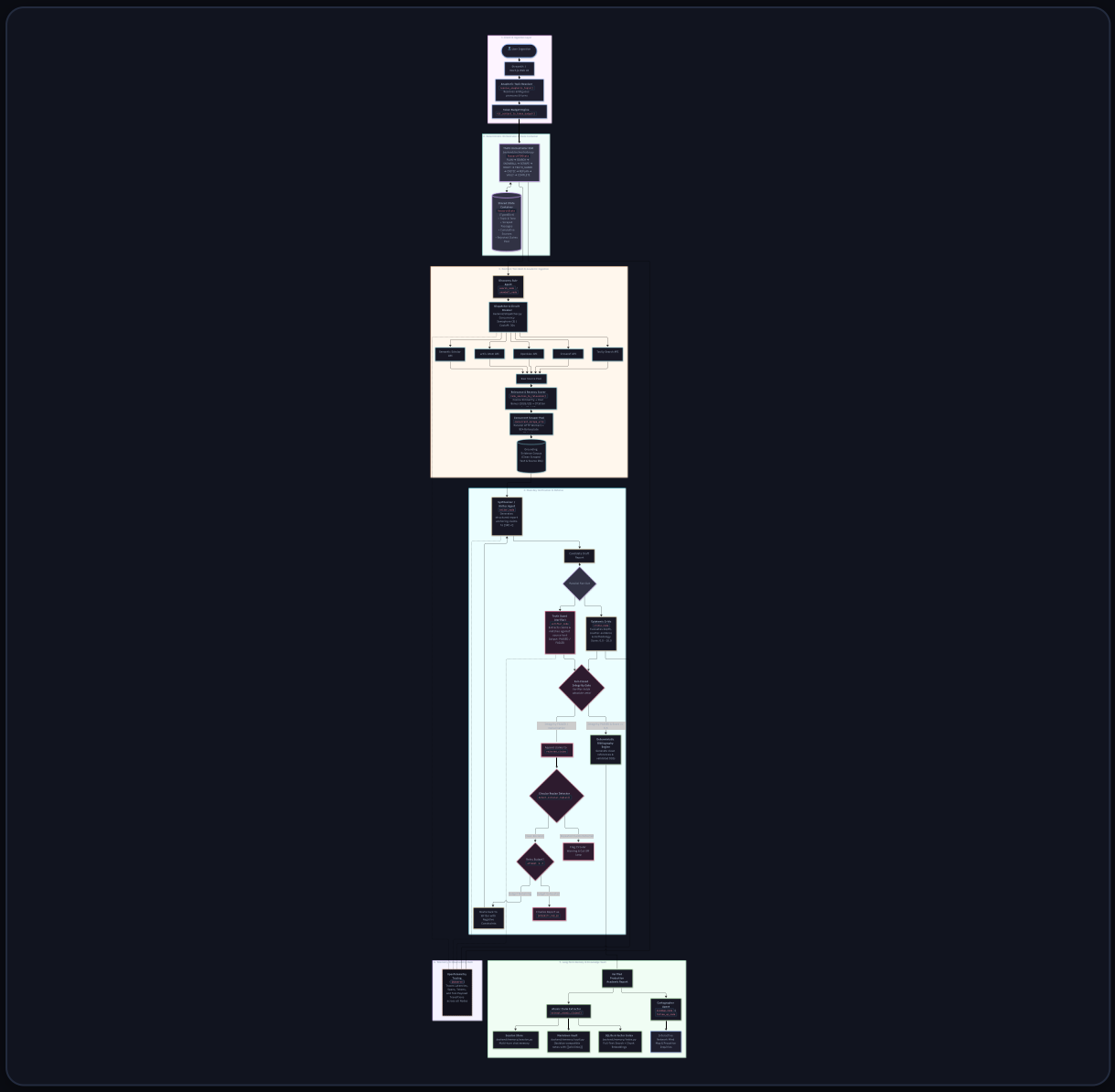
The telemetry, cost control, and error isolation mesh keeping the system stable in production.



diagram

Final Whole Architecture: The Complete Thoth Production Engine

The full, unified end-to-end architecture showing all data flows, sub-agents, security gates, circuit breakers, memory stores, and telemetry.



diagram

Step 7: Irreducible Data Contracts (First Principles AST & State Schemas)

The mathematical and structural primitives that guarantee type safety and clean boundaries across all agent boundaries.



diagram

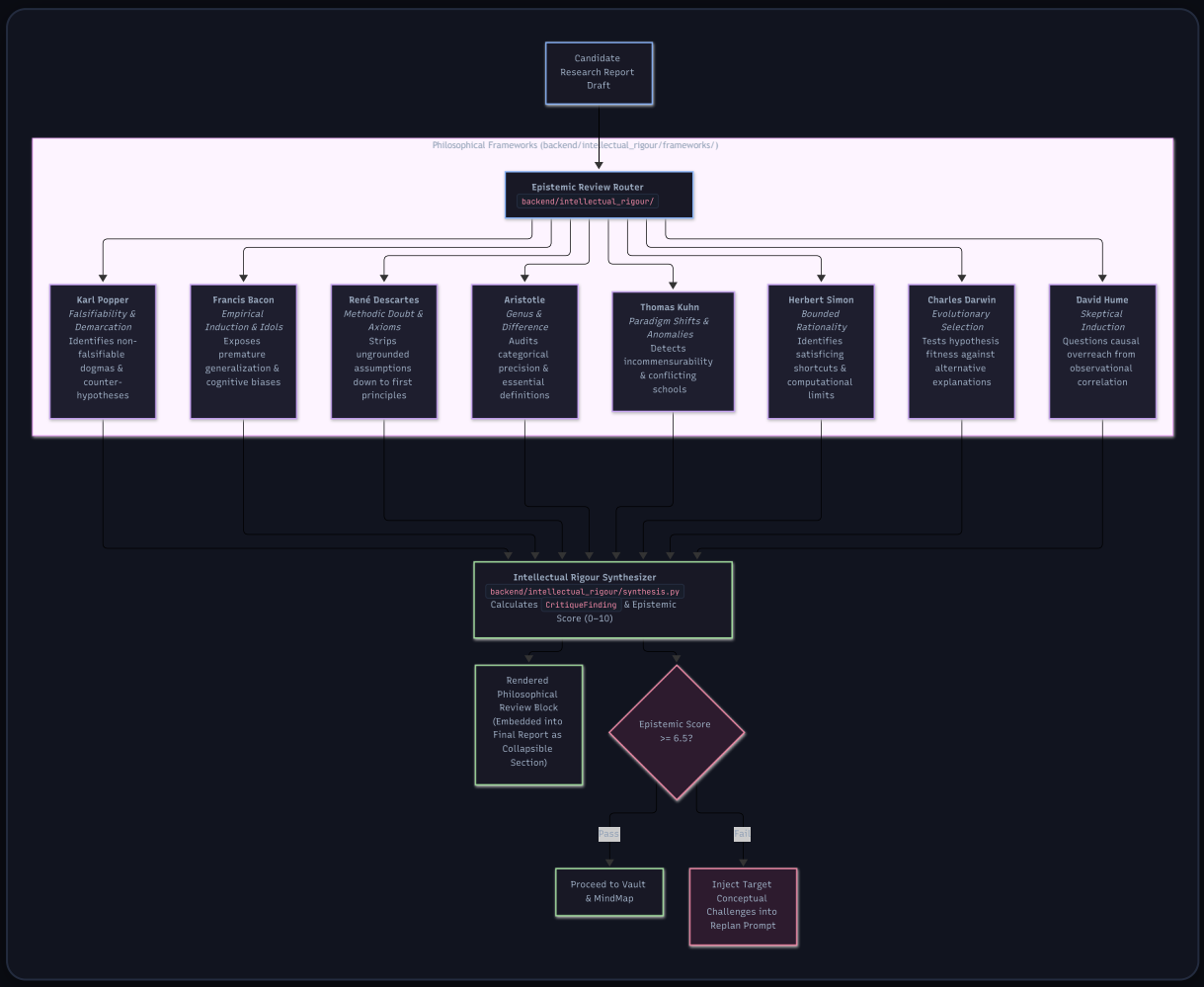
Step 8: Inversion Failure-Mode Matrix (*Man muss immer umkehren*)

A systematic mapping of every catastrophic failure vector and the specific architectural mechanism that guarantees its prevention.

Failure Vector	Catastrophic Manifestation	Root Cause in LLM Systems	Thoth Architectural Inversion Guarantee	Code Anchor
Epistemic Poisoning	Hallucinated citations, phantom DOIs, or fabricated statistics.	LLM autocomplete prioritizes stylistic coherence over factual grounding.	**Fail-Closed NLI Gate**: Claims are extracted into atomic tuples and verified against scraped text. Unverified sources are stripped deterministically.	verifier_node
Circular Replan Deadlock	Agent loops infinitely regenerating the exact same rejected claim in new words.	Negative critique prompt causes slight rephrasing without finding new evidence.	**Circular Replan Detector**: Checks Jaccard and semantic similarity against `rejected_claims`; breaks loop if repeated.	detect_circular_replan
Cascade Quota Exhaustion	429 Rate-Limit bans from Semantic Scholar or arXiv blocking all future turns.	Unbounded concurrent API flooding without backoff or leaky-bucket limits.	**3-State Leaky Bucket Dispatcher**: Hard semaphore cap (3 concurrent requests) + 30s exponential cooloff on 5 failures.	Dispatcher
Longitudinal Context Eviction	System forgets earlier turns and hallucinates conflicting answers.	Raw conversation accumulation overflows LLM token window.	**Deterministic Token Budget Engine**: Sliding window trims middle turns while pinning initial mandate and anaphoric anchors.	fit_context_to_token_budget
Paywall & Scraper Stalls	Pipeline hangs indefinitely on a blocked, rate-limited, or JavaScript-heavy URL.	Unbounded synchronous HTTP requests without timeouts or connection isolation.	**Concurrent Worker Isolation**: URL scraping runs in an async worker pool with a 6-second hard timeout and HTML stripping.	concurrent_scrape_urls
Vault Memory Contamination	Inaccurate claims poison future user conversations through persistent retrieval.	Writing unverified raw agent outputs directly to the long-term knowledge store.	**Post-Verification Persistence Gate**: Vault files (`.md`) and SQLite FTS indexes are updated *only after* the Fail-Closed gate passes.	persist_turn_to_vault

Step 9: The Epistemological Rigour Engine (Philosophic Frameworks Subsystem)

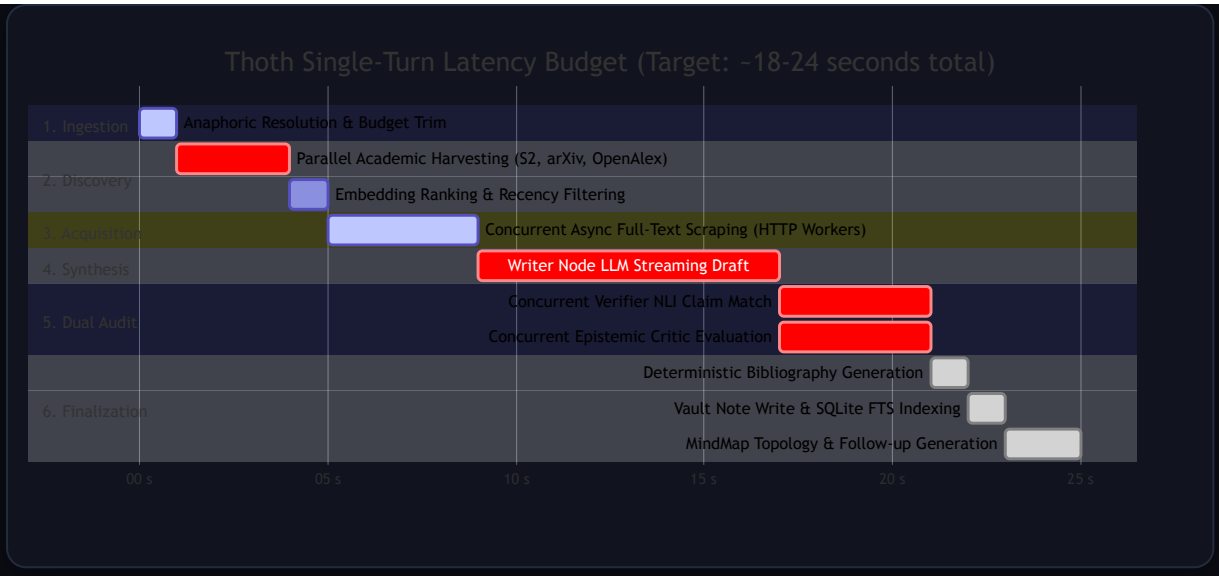
Thoth's multi-perspective epistemic evaluation grid, stress-testing research claims across 10 foundational traditions of scientific philosophy.



diagram

Step 10: Latency, Throughput & Concurrency Budget Profile (Theory of Constraints)

The system's active constraints, temporal budgets, and cybernetic feedback loops.



diagram

Systems Engineering Summary:

1. **The Active Constraint (Goldratt's TOC)**: The primary rate-limiting step is the **Writer Node LLM Generation** (~8s). To elevate this constraint without sacrificing accuracy, **Scraping** and **Evaluation** phases use strict async fan-outs (`asyncio.gather`), running parallel workers rather than sequential calls.
2. **Cybernetic Feedback Control**: Execution timing, tool payloads, and retry events are monitored live via `@observe` spans in `backend/telemetry.py` .
3. **Fail-Closed Stability**: If an external dependency or LLM stalls, circuit breakers trip, preventing resource starvation and guaranteeing that partial reports never masquerade as verified research.